TEXT-TO-IMAGE GENERATION STUDIO

Final Technical Internship Report

Data Science Internship — ElevanceSkills

1 November 2026

Project scope: Diffusion Models • LoRA • BERT • Conditional GANs • Attention • Streamlit

Contents

• Executive Summary

• 1. Project Overview

• 2. Application and System Architecture

• 3. Internship Tasks and Implementation

• 4. Repository Structure and Key Artifacts

• 5. Technical Verification and Evidence

• 6. Technology Stack and Environment

• 7. Running the Application

• 8. Reproducibility, Limitations, and Responsible Use

• 9. Conclusion and Future Work

• Appendix A — Key File References

• Appendix B — Verification Scope Statement

High-level project architecture

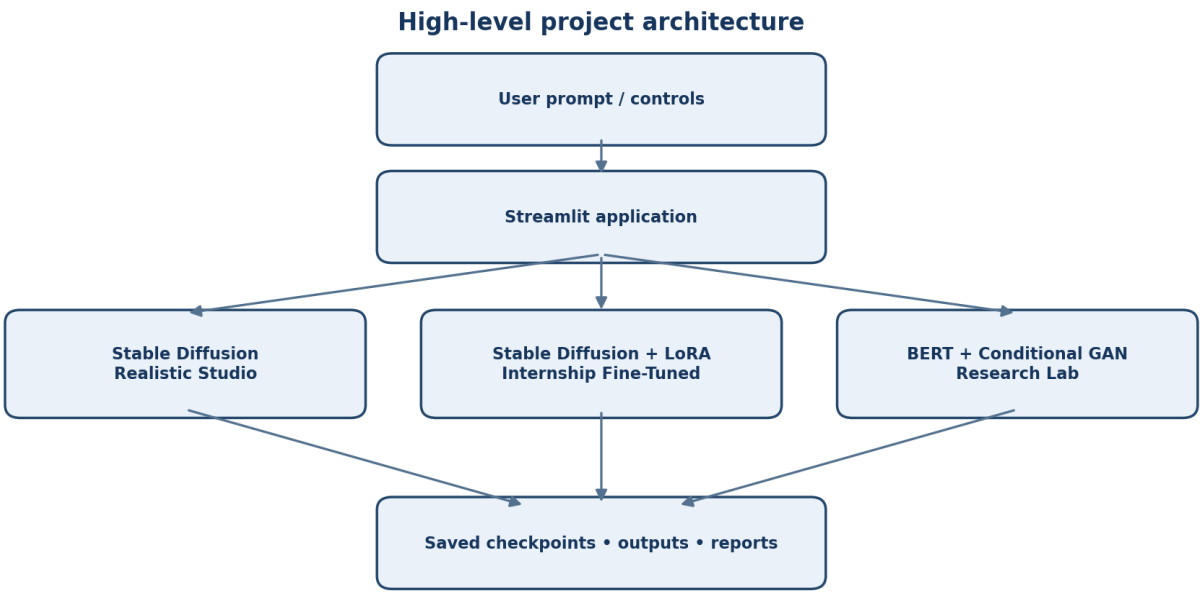


Figure 1. High-level application and model architecture.

Executive Summary

This report documents the design, implementation, experiments, and technical verification of Text-to-Image Generation Studio, an educational and research-oriented project completed as part of a Data Science Internship at ElevanceSkills. The project brings together pretrained text-to-image diffusion, parameter-efficient fine-tuning with Low-Rank Adaptation (LoRA), text preprocessing and BERT embeddings, Conditional Generative Adversarial Networks (Conditional GANs), attention-based GAN experiments, and a Streamlit application.

The repository contains seven Jupyter notebooks, modular application and source-code packages, model checkpoints, numerical experiment artifacts, generated image examples, and project documentation. A final technical verification run reported successful Python compilation, application/source imports, notebook-structure validation, Task 1 LoRA tensor inspection, Task 2 generator forward pass, strict loading of all four Task 5 checkpoints, Task 6 generator forward pass and supporting-artifact reads, dataset-summary inspection, and Git-tracked-file cleanliness.

The verification evidence also identifies boundaries that should be stated transparently: not every notebook has complete saved execution evidence; the Task 1 reproduction script was not independently re-run end-to-end in the current review; and the diffusion pipelines depend on external pretrained model availability, compatible dependencies, and adequate compute resources.

1. Project Overview

Text-to-Image Generation Studio explores how natural-language descriptions can be transformed into image-generation conditions and used by different generative approaches. It combines general-purpose pretrained diffusion pipelines with smaller experimental GAN workflows, and exposes selected capabilities through a unified Streamlit interface.

Project objectives

• Explore image-caption dataset structure and descriptive statistics.

• Preprocess natural-language text and generate contextual BERT representations.

• Implement and inspect text-conditioned Conditional GAN workflows.

• Compare baseline and attention-based GAN experiment artifacts.

• Demonstrate parameter-efficient adaptation of a pretrained Stable Diffusion model using LoRA.

• Integrate selected model workflows into a Streamlit application.

• Preserve notebooks, checkpoints, generated examples, and documentation for review and further development.

Scope and interpretation

The project contains distinct model families with different goals. Stable Diffusion is used for general-purpose image generation, while the GAN research pipeline is a low-resolution experimental implementation. Their outputs should not be compared as if they were equivalent systems.

2. Application and System Architecture

The Streamlit application provides three primary generation modes: Realistic Studio, Internship Fine-Tuned, and Research Lab. Supporting interfaces include Attention Explorer and project-information content. The application delegates model loading and generation to a model-service layer, while reusable machine-learning code is maintained under src/.

Layer / component	Responsibility
Application (app/)	Streamlit interface, configuration, user controls, and model-service integration.
Model service (app/services/)	Model availability checks, checkpoint loading, and generation dispatch.
Reusable source (src/)	GAN architectures, diffusion/LoRA utilities, text processing, data loading, and image utilities.
Notebooks (notebooks/)	Stepwise experimentation and research record for the internship tasks.
Models (models/)	Selected Task 5 and Task 6 checkpoints and related model artifacts.
Outputs (outputs/)	LoRA adapter, generated examples, dataset summaries, and GAN comparison artifacts.
Entry point (run_app.py)	Root-level application launch entry point.

3. Internship Tasks and Implementation

Task 1 — Pretrained Model Adaptation with LoRA

This task explores parameter-efficient fine-tuning of a pretrained Stable Diffusion model using LoRA. The repository retains the adapter at outputs/task1/lora_custom_dataset/pytorch_lora_weights.safetensors, along with baseline and adapted example images, a comparison image, and a training-loss plot.

Verification evidence: the saved Safetensors file was successfully read and contained 256 tensors; all inspected tensors were finite. This confirms that the adapter file is readable and its tensors passed the reported finite-value check. It does not independently prove that a complete Stable Diffusion inference session with the adapter succeeds, nor that fine-tuning improves image quality for all prompts.

Task 2 — Conditional Generative Adversarial Network

The Conditional GAN work explores image generation conditioned on text or class-related information. The reported generator forward-pass check produced a tensor of shape (2, 1, 64, 64), and the check exercised square/circle labels. This is evidence of a successful forward pass for the tested configuration, not a broad quality evaluation.

Task 3 — Text Preprocessing and BERT Embeddings

This task covers cleaning and tokenizing text descriptions and representing them with contextual embeddings from BERT. The project configuration identifies bert-base-uncased and a 768-dimensional embedding size. Text representations provide conditioning information for the research workflow. The notebooks and reusable source modules document the implementation; notebook execution evidence is summarized in Section 5.

Task 4 — Public Dataset Analysis

The project includes Flickr8k-oriented dataset analysis and a saved summary at outputs/graphs/flickr8k_analysis_results.csv. The final verification script reported that the summary was readable and contained 21 rows. The earlier project documentation reports 8,000 images split into 6,000 training, 1,000 validation, and 1,000 test images, with five captions per image. These dataset statistics are reported project figures and should be interpreted according to the actual dataset version and analysis procedure used.

Task 5 — Attention Mechanism Experiments

The attention experiments compare saved baseline and attention-based GAN artifacts. The outputs/comparisons/ directory includes generator and discriminator loss arrays, generated sample tensors, and attention weights. The models/cgan/task5/ directory contains generator and discriminator checkpoints, configuration, model code, metrics, validation report, and visual evidence.

Verification evidence: all four Task 5 checkpoints—attention generator, baseline generator, attention discriminator, and baseline discriminator—were reported to load with strict state-dictionary matching. Forward-pass checks were also reported as passed. This validates compatibility between the tested model definitions and checkpoint keys; it does not establish that attention outperforms the baseline.

Task 6 — Integrated Text-Conditioned Generation Pipeline

The integrated research pipeline connects a natural-language prompt to text preprocessing, BERT-based representation, and a Conditional GAN generator. The configured research output is a 64 × 64 grayscale image. The relevant generator, discriminator, embeddings, and loss arrays are stored under models/cgan/.

Verification evidence: the Task 6 generator loaded and completed a forward pass with reported output shape (2, 1, 64, 64). The discriminator checkpoint was readable as an OrderedDict, the embeddings artifact was readable as a dictionary, and both loss arrays contained 25 finite values.

4. Repository Structure and Key Artifacts

The repository is organized to separate application code, reusable modules, experiments, checkpoints, and outputs. The principal tracked structure is summarized below; individual directories may contain additional files.

```
Text-to-Image-Generator/
■■■ app/
■ ■■■ app.py
■ ■■■ config.py
■ ■■■ services/model_service.py
■■■ data/ # dataset documentation
■■■ models/
■ ■■■ cgan/
■ ■■■ task5/ # attention/baseline GAN artifacts
■ ■■■ task6_text_conditioned_generator.pth
■ ■■■ task6_text_conditioned_discriminator.pth
■ ■■■ task6_text_embeddings.pt
■ ■■■ task6_*_losses.npy
■■■ notebooks/ # seven numbered notebooks
■■■ outputs/
■ ■■■ comparisons/
■ ■■■ generated_images/
■ ■■■ graphs/
■ ■■■ task1/ # LoRA adapter and examples
■■■ reports/
■■■ scripts/
■ ■■■ reproduce_task1_lora.py
■■■ src/
■■■ requirements.txt
■■■ run_app.py
■■■ README.md
```

Artifact	Repository location / purpose
Task 1 LoRA adapter	outputs/task1/lora_custom_dataset/pytorch_lora_weights.safetensors
Task 5 checkpoints/configuration	models/cgan/task5/
Task 6 checkpoints and embeddings	models/cgan/
Task 5 comparison arrays and samples	outputs/comparisons/
Dataset analysis summary	outputs/graphs/flickr8k_analysis_results.csv
Generated showcase images	outputs/generated_images/
Notebook experiments	notebooks/00 through notebooks/06

5. Technical Verification and Evidence

The following results were reported by the final project verification run supplied during the review. They are presented as observed checks, not as a claim that every possible runtime path or model mode has been exhaustively tested.

Verification area	Reported result	Interpretation
Python source compilation	PASS — app, src, scripts	Python source files compiled successfully.
Application/source imports	PASS — 10 modules imported	The tested modules imported in the verification environment.
Notebook structure and saved outputs	PASS — 7 valid notebooks; 0 saved errors	Notebook JSON structure and saved output records were inspected.
Task 1 LoRA artifact	PASS — 256 tensors; all finite	Safetensors file readable and tensor finite-value check passed.
Task 2 CGAN forward pass	PASS — (2, 1, 64, 64)	Tested generator forward pass completed.
Task 5 checkpoints	PASS — four strict loads	Both generator and discriminator checkpoint pairs matched their tested model definitions.
Task 6 generator	PASS — (2, 1, 64, 64)	Generator loaded and completed tested forward pass.
Task 6 supporting artifacts	PASS	Discriminator and embeddings readable; both loss arrays had 25 finite values.
Dataset summary / Task 5 validation	PASS	CSV summary reported as 21 rows; Task 5 validation report marked PASS.
Git tracked-file cleanliness	PASS — 74 tracked files	No tracked backup, cache, or virtual-environment files were reported.

Notebook execution evidence

Notebook	Code cells	Executed	Saved errors
00_environment_setup.ipynb	10	6	0
01_dataset_analysis.ipynb	15	13	0
02_text_preprocessing_embeddings.ipynb	16	2	0
03_conditional_gan.ipynb	20	20	0
04_attention_mechanism.ipynb	29	29	0
05_pretrained_model_fine_tuning.ipynb	17	0	0
06_integrated_text_to_image_pipeline.ipynb	34	34	0

The notebook review found seven structurally valid notebooks and no saved error outputs. However, execution counts show that notebooks 02 and 05 do not have complete saved execution evidence, and notebook 05 has no executed code cells recorded. A notebook with no saved error output is not necessarily a fully executed notebook. These gaps should remain explicit in the final submission.

Additional checks reported during the review

• python -m pip check reported: No broken requirements found.

• Python compileall completed successfully for app, src, and scripts.

• Task 5 attention generator was independently loaded with strict state-dictionary matching.

• Git ignore checks confirmed representative backup files were ignored.

• Git status showed a modified .gitignore and notebook, plus untracked report files at the time of that check; these changes require intentional review and staging before a final commit.

6. Technology Stack and Environment

Technology	Use in the project
Python	Primary implementation language.
PyTorch / Torchvision	Deep-learning models, tensor operations, and image transforms.

Technology	Use in the project
Streamlit	Interactive application interface.
Hugging Face Diffusers	Pretrained diffusion pipelines.
Transformers	BERT tokenizer and text encoder.
PEFT / Safetensors	LoRA adapter integration and serialized adapter weights.
NumPy / Pandas	Numerical experiment arrays and dataset analysis.
Pillow / Matplotlib	Image handling and experiment visualizations.
Jupyter	Notebook-based experimentation.

The requirements file uses bounded dependency ranges for the primary libraries. The project notes previously recorded a Windows development environment with Python 3.12.10, PyTorch 2.9.1+cu130, Diffusers 0.40.0, Streamlit 1.64.0, and an NVIDIA RTX 3050 with 6 GB VRAM. These are reported development-environment details, not a guarantee that all supported dependency combinations or hardware configurations are equivalent.

7. Running the Application

From the repository root in Windows PowerShell, activate the project virtual environment and launch Streamlit:

```
.\.venv\Scripts\Activate.ps1
python -m streamlit run .\run_app.py
```

Open the local address printed by Streamlit in the terminal. Keep the terminal session active while using the application; stop it with Ctrl+C. A successful Streamlit launch does not by itself prove that every model has loaded or that every generation mode can complete inference.

Stable Diffusion modes may require downloading pretrained weights and can require substantial GPU memory. The GAN research mode uses local checkpoints. Actual availability depends on the paths, package versions, model identifiers, cache state, and hardware.

8. Reproducibility, Limitations, and Responsible Use

Reproducibility

• Use the documented environment and install dependencies from requirements.txt in a clean virtual environment.

• Record the dataset source/version, model identifier or revision, random seed, prompt, and generation settings for each experiment.

• Keep baseline and adapted comparisons controlled by using the same prompts and comparable generation settings.

• Back up existing Task 1 outputs before running scripts that may overwrite files.

• Treat saved checkpoints and outputs as evidence of artifacts; establish reproducibility through a fresh run where feasible.

Known limitations

• The Conditional GAN research pipeline produces 64 × 64 grayscale outputs and is not intended to match the visual fidelity of large diffusion models.

• LoRA behavior depends on the base model, adapter compatibility, training data, and inference settings; improvement is not guaranteed for every prompt.

• Saved attention weights, loss curves, and samples do not alone prove that attention improves generation quality.

• Some notebook execution records are incomplete, as detailed in Section 5.

• The Task 1 reproduction script was not independently revalidated end-to-end in the current technical review.

• Pretrained model downloads, licensing, storage, and GPU-memory constraints may affect setup and execution.

Responsible use

Users should respect the licenses and usage conditions of datasets, pretrained models, and redistributed checkpoints. Do not commit credentials, private tokens, or restricted dataset content. Generated images should be reviewed for unsuitable, misleading, or biased outputs before external use.

9. Conclusion and Future Work

Text-to-Image Generation Studio provides an integrated educational project spanning diffusion-based generation, LoRA adaptation, text representation, Conditional GANs, attention experiments, and application development. The supplied verification evidence confirms that the main source packages compile and import, selected GAN checkpoints load strictly, tested forward passes complete, and key supporting artifacts are readable.

The strongest next steps are to complete and preserve reproducible notebook runs for the partially executed notebooks, revalidate the Task 1 training/reproduction path in a controlled environment, exercise each Streamlit generation mode end-to-end, and record prompt-controlled quantitative or qualitative evaluation where model comparisons are claimed. A clean final Git review and a short application demonstration will strengthen the handover.

Appendix A — Key File References

• README.md — project overview, setup, features, and repository guide.

• requirements.txt — bounded Python dependency declarations.

• run_app.py — Streamlit application entry point.

• app/app.py — application interface and mode selection.

• app/services/model_service.py — model loading and generation service.

• app/config.py — project paths and model configuration.

• scripts/reproduce_task1_lora.py — Task 1 reproduction script.

• models/README.md, data/README.md, scripts/README.md — directory-level documentation.

Appendix B — Verification Scope Statement

This report is based on the repository structure, command outputs, and verification results shared during the project review. The checks listed as PASS reflect the supplied verification run. They should not be interpreted as independent execution by the report author, exhaustive testing of all prompts or hardware, formal model-quality benchmarking, or confirmation of external dataset/model licensing. Any final submission should retain the verification log and distinguish completed tests from planned follow-up work.